Guild SA Buildathon 01

Challenge and Architecture Guidelines

Theme: AI-Powered Applications and Systems

Welcome to Guild SA Buildathon 01.

Over the next 10 hours, your team must design, build, and demonstrate a functional Minimum Viable Product that uses open-weight artificial intelligence to solve one clearly defined real-world problem.

Each team must select one of the four challenges in this document.

The goal is not to build a production-ready platform. Judges are looking for a focused MVP that demonstrates:

- A complete and working user journey
- Meaningful use of an AI model
- Reasoning beyond basic text extraction or summarisation
- Reliable structured output
- Appropriate handling of missing or ambiguous information
- An interface suitable for the intended user
- Human review of AI-generated results
- A clear connection to the selected problem

A successful submission should do more than call an AI model and display its response. It should use AI as part of a practical decision-support workflow.

---

1. Buildathon Objective

Your solution should follow this general workflow:

User input

↓

AI extraction and interpretation

↓

Validation and ambiguity detection

↓

Recommendation or intelligent action

↓

Explanation and confidence indication

↓

Human review and correction

↓

Approved structured output

Every solution must demonstrate that the AI can:

1. Understand an unstructured input.
2. Convert it into structured information.
3. Identify missing, conflicting, or suspicious information.
4. Recommend or perform an appropriate next step.
5. Explain the basis for its decision.
6. Allow a human user to review the result.

---

2. Technical Guardrails

2.1 Approved AI infrastructure

To keep the competition accessible, teams may not rely on paid proprietary AI APIs for the core functionality being evaluated.

Teams must use an open-source or open-weight model through an accessible inference platform, such as:

- GroqCloud
- Hugging Face Inference
- Ollama or another local model runtime

- Another organiser-approved open-weight inference provider

The organisers will confirm approved providers and available models before the event. Free-tier limits and model availability may change.

Teams may use any reasonable frontend and backend technology.

2.2 Core technical requirements

Every solution must:

- Send user-provided or supplied synthetic data to a live or locally running AI model.
- Display the processed result inside the application.
- Request or produce structured output.
- Handle missing, invalid, contradictory, or ambiguous information.
- Generate an explanation for important recommendations.
- Include a human review step.
- Allow the user to edit or correct relevant AI-generated values.
- Avoid exposing API keys in frontend source code.
- Provide a clear error message if the AI service is unavailable.
- Include at least three working demonstration cases.

At least one demonstration case must contain incomplete, ambiguous, duplicate, or conflicting information.

2.3 Responsible AI requirements

The application must not silently invent missing information.

When information cannot be found, the system should:

- Return `null`, `unknown`, or an equivalent value.
- Identify the missing field.
- Ask a relevant clarification question where appropriate.
- Prevent high-risk actions from being automatically confirmed.
- Allow a human user to approve or reject the result.

Confidence scores may be used to communicate uncertainty. Teams should explain how their confidence value is produced and must not present it as a scientifically calibrated probability unless they have implemented an actual calibration method.

2.4 Data requirements

Teams must use the synthetic data provided by the organisers or create additional fictional data.

Teams may not use:

- Real customer information
- Real driver or student information
- Private municipal reports
- Real banking credentials
- Personally identifiable information
- Confidential partner information

No production-system integration is required.

---

3. Suggested Architecture

Teams are free to design their own architecture. A suitable MVP architecture may contain the following components:

Frontend interface

↓

Backend or secure server function

↓

Input validation

↓

Open-weight AI model

↓

Structured-output validation

↓

Business rules or deterministic calculations

↓

Human review screen

↓

Final approved record

3.1 Recommended separation of responsibilities

AI model

The AI model may be used for:

- Understanding unstructured language
- Extracting relevant information
- Identifying ambiguity
- Classifying incidents or expenses
- Producing explanations
- Generating checklists and plans
- Recommending appropriate next actions

Application code or rule engine

Normal application code should be used where possible for:

- Required-field validation
- Arithmetic
- VAT calculations
- Duplicate comparisons
- Date validation
- Vehicle-capacity comparisons
- Sorting and filtering
- Schema validation
- Final approval status

Students should not use an LLM to perform arithmetic that can be calculated reliably in code.

3.2 Suggested structured response

Teams may adapt the following response format:

```
{

  "extracted_data": {},

  "status": "requires_review",
```

```
  "missing_fields": [],
  "conflicts": [],
  "recommended_action": "",
  "reasoning": "",
  "clarification_questions": [],
  "confidence": 0.82,
  "human_approval_required": true
}
```

Equivalent structures are acceptable.

---

4. The Four Buildathon Challenges

Challenge 1: VelozTech Intelligent Fleet and Dispatch Assistant

4.1 Industry context

This challenge is inspired by VelozTech's VelozTrack-Us Fleet Management System.

VelozTrack-Us supports fleet operations such as:

- Trip management
- Cargo manifests
- Vehicle and driver management
- Pickup and delivery workflows
- Driver updates
- Proof-of-delivery records
- Operational notifications

Teams are not expected to reproduce the VelozTrack-Us platform. The challenge is to develop a small AI decision-support feature that could complement a fleet-management workflow.

4.2 The problem

Fleet managers and dispatchers receive cargo and delivery information through:

- Dispatcher messages
- Cargo manifests
- Emails
- Driver updates
- Delivery instructions
- Proof-of-delivery notes

These messages may be incomplete, informal, or contradictory.

Important information such as cargo weight, vehicle capacity, pickup time, delivery location, reference number, contact details, or drop-off bay may be missing.

Fleet managers must identify these problems and decide what should happen next.

4.3 The challenge

Build an AI-powered fleet and dispatch assistant that:

1. Converts an unstructured fleet message into structured trip information.
2. Identifies missing or conflicting information.
3. Detects at least one operational risk.
4. Recommends an appropriate next action.
5. Explains the recommendation.
6. Allows a dispatcher to edit and approve the result.

4.4 Required MVP workflow

The user must be able to:

1. Paste a cargo manifest, dispatcher message, driver update, or delivery note.
2. Submit it for analysis.
3. View the extracted trip information.
4. View missing fields, conflicts, or operational risks.
5. View a recommended next action.
6. View the reason for the recommendation.
7. Review and edit the extracted information.
8. Approve or reject the final result.
9. View a structured dispatcher log entry.

4.5 Required AI output

The solution must extract at least four applicable fields:

- Trip or load reference
- Cargo description
- Cargo weight
- Cargo quantity
- Pickup location
- Pickup time
- Drop-off location
- Delivery time
- Vehicle capacity
- Driver instruction
- Contact details
- Drop-off bay or gate

The system must also produce:

- A short trip summary
- A driver or dispatcher checklist
- Missing-information warnings
- A recommended next action
- A justification for the recommendation
- A clarification question when necessary
- A chronological operational log entry

4.6 Example input

> Manifest VT-2063. Pick up 18 steel coils, total weight 6,500 kg, from Isando Metals at 07:00. Assigned vehicle capacity is 5,000 kg. Deliver to Rosslyn Fabrication Plant, Gate 2.

4.7 Example output


```
{

  "trip_reference": "VT-2063",

  "cargo_description": "Steel coils",

  "cargo_quantity": 18,

  "cargo_weight_kg": 6500,
```

```
  "pickup_location": "Isando Metals",
  "pickup_time": "07:00",
  "dropoff_location": "Rosslyn Fabrication Plant",
  "dropoff_gate": "Gate 2",
  "assigned_vehicle_capacity_kg": 5000,
  "status": "requires_review",
  "risk_level": "high",
  "conflicts": [
    "Cargo exceeds the assigned vehicle capacity by 1,500 kg."
  ],
  "recommended_action": "Reassign a suitable vehicle or split the load.",
  "reasoning": "The assigned vehicle cannot legally or safely carry the stated cargo weight.",
  "driver_checklist": [
    "Do not begin loading.",
    "Notify the dispatcher of the capacity conflict.",
    "Wait for an approved replacement vehicle or split-load instruction."
  ],
  "clarification_questions": [],
  "confidence": 0.98,
  "human_approval_required": true,
  "log_entry": "Capacity conflict detected for trip VT-2063. Dispatch approval required before loading."
}
```


4.8 Accepted recommendations

Depending on the input, recommendations may include:

- Delay departure
- Request missing cargo information
- Reassign a suitable vehicle
- Reassign a refrigerated vehicle
- Notify the customer
- Update a delivery instruction
- Escalate a cargo discrepancy
- Pause pickup verification
- Request a complete proof of delivery
- Escalate an incident to the dispatcher

Every important recommendation must include a reason.

4.9 Scope boundary

Teams are not required to:

- Rebuild VelozTrack-Us
- Implement live GPS tracking
- Calculate live routes
- Build a driver mobile application
- Integrate with VelozTech's production API
- Implement authentication or multiple user roles
- Use real company, vehicle, cargo, or driver data

A standalone web application using the supplied synthetic fleet data is sufficient.

4.10 Optional innovation features

Teams may implement one or more of the following:

- Voice-based driver updates
- Multilingual driver-message interpretation
- Comparison with a previous trip message
- Context memory across trip updates
- Visual trip-event timeline
- Risk-level dashboard

Challenge 2: Hyper-Local Utility Reporting Assistant

5.1 The problem

Community members regularly encounter:

- Water leaks
- Electricity outages
- Potholes
- Broken traffic lights
- Illegal dumping
- Sewage overflows
- Fallen trees
- Missing manhole covers

Residents may describe the same incident differently. Municipal staff must determine:

- What happened
- Where it happened
- How urgent it is
- Whether it has already been reported
- Which department should receive it

5.2 The challenge

Build an AI-powered utility reporting assistant that converts informal community complaints into structured service tickets.

The assistant must also compare reports, detect possible duplicates, and route incidents to the appropriate municipal department.

5.3 Required MVP workflow

The user must be able to:

1. Enter a plain-language utility complaint.
2. Submit it for AI analysis.
3. View the detected incident category.
4. View the extracted location.
5. View the urgency level.

6. View the assigned municipal department.
7. Compare the report with a small list of existing reports.
8. View a possible duplicate match.
9. Review the AI's explanation.
10. Confirm whether the report should create a new incident or join an existing incident.
11. Approve the final service ticket.

5.4 Required AI output

The system must produce:

- Incident category
- Location description
- Urgency level
- Short incident summary
- Assigned department
- Department-routing reason
- Possible duplicate incident
- Duplicate-matching explanation
- Missing information
- Clarification question where necessary
- Structured service ticket

5.5 Minimum department routing

Incident type	Suggested department
Water leak or burst pipe	Water and Sanitation
Electricity outage	Electricity or Energy
Pothole or damaged road	Roads and Transport
Broken traffic signal	Roads and Transport

Illegal dumping	Waste Management
Sewage overflow	Water and Sanitation
Fallen tree	Parks or Environmental Management
Missing manhole cover	Roads, Water or relevant infrastructure department

Equivalent department names are acceptable.

5.6 Duplicate incident example

Existing reports

> There is a burst pipe on Main Road outside the Spar.
>
> Water is flooding the road near the Main Road traffic lights.

New report

> Huge water leak opposite the Spar near the traffic lights.

Expected result


```
{
  "category": "burst_water_pipe",
  "location": "Main Road near the Spar and traffic lights",
  "urgency": "high",
  "assigned_department": "Water and Sanitation",
  "routing_reason": "The report describes a municipal water leak.",
  "possible_duplicate": true,
```

"matched_incident_ids": [

"INC-104",

"INC-107"

],

"duplicate_reasoning": "The reports describe a water leak in the same Main Road landmark area.",

"recommended_action": "Add the new report to the existing incident and increase the report count.",

"human_approval_required": true

}

The user must be able to confirm or reject the duplicate suggestion.

5.7 Ambiguity requirement

If a report says:

> Something smells bad near the park. Please send someone.

The system should not confidently invent a location or incident type. It should request clarification, for example:

- Which park?
- Which suburb or municipality?
- Does the smell appear to come from sewage, waste, smoke, or chemicals?
- When was it first noticed?

5.8 Scope boundary

Teams are not required to:

- Connect to a real municipality
- Dispatch emergency services
- Generate exact GPS coordinates
- Implement a city-wide geographic information system
- Process real citizen information
- Build a production ticketing system

A supplied list of synthetic reports is sufficient for duplicate detection.

5.9 Optional innovation features

Teams may implement:

- Interpretation of one additional South African language
- Map-based visualisation using simulated locations
- Incident heat maps
- Context memory across citizen follow-ups
- Notifications when multiple reports are linked
- Image-based incident categorisation

Challenge 3: Academic Study and Assessment Co-Pilot

6.1 The problem

Students often struggle to interpret:

- Module guides
- Assignment briefs
- Assessment rubrics
- Project requirements
- Deadlines
- Formatting instructions
- Submission restrictions

A basic summary does not always help a student decide how to complete the work.

Students need an assistant that can convert academic requirements into an actionable checklist and realistic study plan without inventing requirements that are not present in the source document.

6.2 The challenge

Build an AI-powered academic co-pilot that analyses an assessment brief and produces:

- An assessment checklist

- Available marking information
- Common mistakes
- A recommended work plan
- A personalised study schedule

6.3 Required MVP workflow

The user must be able to:

1. Paste academic instructions or upload one supported text-based document.
2. Submit the document for analysis.
3. View the extracted assessment requirements.
4. View deadlines and restrictions.
5. View a submission checklist.
6. View the marking breakdown, where supplied.
7. View common mistakes or risks.
8. Enter basic planning information.
9. Generate a recommended study plan.
10. Review and edit the plan.

6.4 Student planning inputs

The application should request at least two of the following:

- Available study hours per week
- Preferred study days
- Self-reported difficulty
- Current completion percentage
- Desired completion date
- Other academic commitments

6.5 Required AI output

The system must produce:

- Assessment title
- Due date
- Required deliverables
- Formatting requirements
- Prohibited actions
- Submission checklist
- Marking breakdown, if provided
- Missing or ambiguous instructions

- Common mistakes
- Recommended work plan
- Study milestones
- Estimated workload
- Revision or final-review period

6.6 Important reliability rule

If the document does not provide marks or weighting, the system must not invent a marking breakdown.

It should return:

```
{
  "marking_breakdown": null,
  "warning": "The supplied brief does not contain mark allocations."
}
```

The system may still create a task-priority plan, but it must clearly label it as a recommendation rather than an official marking guide.

6.7 Example input

> Teams of three must design a working campus navigation prototype. Submit one PDF report and a Git repository link by 16:00 on 28 August 2026. The report may not exceed 2,500 words and must include a problem statement, user stories, architecture diagram, test evidence and individual reflection. A five-minute demonstration will take place on 31 August.

Student availability:

```
{
  "available_hours_per_week": 8,
  "preferred_days": [
    "Tuesday",
    "Thursday",
    "Saturday"
```

```
  ],
  "self_reported_difficulty": "medium"
}
```

6.8 Example output

```
{
  "assessment_title": "Campus Navigation Prototype",
  "work_mode": "Team of three",
  "submission_deadline": "28 August 2026 at 16:00",
  "demonstration_date": "31 August 2026",
  "deliverables": [
    "PDF report",
    "Git repository link",
    "Five-minute demonstration"
  ],
  "requirements": [
    "Maximum 2,500 words",
    "Problem statement",
    "User stories",
    "Architecture diagram",
    "Test evidence",
    "Individual reflection"
  ],
  "marking_breakdown": null,
  "warnings": [
```

```
  "No mark allocation was supplied."
],
"common_mistakes": [
  "Exceeding the word limit",
  "Forgetting the individual reflection",
  "Submitting the report without the repository link",
  "Leaving testing until the final day"
],
"study_plan": [
  {
    "milestone": "Define the problem and user stories",
    "estimated_hours": 2
  },
  {
    "milestone": "Build the core prototype",
    "estimated_hours": 6
  },
  {
    "milestone": "Create architecture diagram and test evidence",
    "estimated_hours": 3
  },
  {
    "milestone": "Complete and review the report",
    "estimated_hours": 4
```

```
    },
    {
      "milestone": "Rehearse the demonstration",
      "estimated_hours": 2
    }
  ],
  "human_review_required": true
}
```

6.9 Scope boundary

Teams are not required to:

- Write an assignment on behalf of a student
- Grade academic work
- Detect plagiarism
- Integrate with a learning-management system
- Support every file format
- Generate official marks that are not supplied
- Process real student records

Supporting pasted text and one uploaded text-based format is sufficient.

6.10 Optional innovation features

Teams may implement:

- Draft checking against extracted requirements
- Context memory when a student updates availability
- Calendar export
- Progress tracking
- Study reminders
- Short revision quiz
- Accessibility features such as text-to-speech

Challenge 4: Micro-Business Financial Workflow Assistant

7.1 The problem

Small-business owners and freelancers spend significant time:

- Capturing receipts
- Categorising expenses
- Calculating VAT
- Detecting duplicate transactions
- Checking invoice calculations
- Reviewing spending patterns

Basic extraction is useful, but business owners also need help understanding what their transaction history means.

7.2 The challenge

Build an AI-powered financial workflow assistant that converts receipt or invoice text into structured ledger records and identifies at least one meaningful financial pattern or anomaly.

7.3 Required MVP workflow

The user must be able to:

1. Paste or load multiple synthetic receipt or invoice records.
2. Submit the records for processing.
3. View extracted transaction information.
4. View VAT calculations.
5. View transactions in a simple ledger.
6. View detected anomalies.
7. View at least one financial insight across the records.
8. View the evidence supporting the insight.
9. Correct transaction information.
10. Approve the final ledger entries.

7.4 Required transaction output

For every transaction, the system must extract:

- Merchant or supplier
- Transaction date, when provided
- Description or line items
- Total amount
- VAT status
- VAT amount
- Expense category
- Payment method, when provided
- Missing information
- Anomaly status

7.5 VAT calculation

For a VAT-inclusive total, calculate the VAT portion as:

VAT portion = VAT-inclusive total × 15 ÷ 115

For example:

R805.00 × 15 ÷ 115 = R105.00 VAT

VAT and other arithmetic should be calculated in application code rather than relying only on the language model.

7.6 Required financial intelligence

The application must identify at least one of the following:

- Duplicate transaction
- VAT mismatch
- Line-item total mismatch
- Missing merchant or date
- Unusual transaction amount
- High spending concentration in one category
- Change in category spending across supplied periods
- Recurring subscription
- Large cash transaction relative to other records

Every insight must reference the records supporting it.

7.7 Example output

{

```
  "transaction_id": "EXP-008",
  "merchant": "MZANSI OFFICE MART",
  "date": "2026-07-14",
  "total": 805.00,
  "vat": 105.00,
  "category": "office_supplies",
  "anomaly": {
    "type": "probable_duplicate",
    "matched_transaction_id": "EXP-001",
    "reasoning": "The merchant, date, line items, total and payment method are identical.",
    "confidence": 0.99
  },
  "recommended_action": "Review both transactions before adding EXP-008 to the ledger.",
  "human_approval_required": true
}
```


A cross-transaction insight might be:


```
{
  "insight": "Office-supply spending may contain a duplicate transaction.",
  "supporting_transactions": [
    "EXP-001",
    "EXP-008"
  ],
  "financial_effect": "Removing the duplicate would reduce recorded expenses by R805.00.",
  "recommended_action": "Confirm whether both payments occurred before finalising the ledger."
```

}

7.8 Reliability requirements

The system must:

- Distinguish between VAT-inclusive and VAT-exclusive amounts.
- Avoid calculating VAT when VAT status is unknown unless clearly labelled as an estimate.
- Identify missing merchants or dates.
- Show the transactions supporting each insight.
- Allow the user to correct extracted values.
- Avoid presenting the prototype as professional tax advice.

7.9 Scope boundary

The solution is an educational prototype.

Teams are not required to:

- Submit information to SARS
- Provide tax or accounting advice
- Connect to a bank
- Process real financial information
- Implement complete bookkeeping
- Support receipt-image OCR
- Build forecasting or a full accounting platform

Text-based synthetic transaction data is sufficient.

7.10 Optional innovation features

Teams may implement:

- Spending visualisations
- Monthly category comparisons
- Percentage-change calculations
- Recurring-subscription detection
- Cash-flow warnings
- Context memory across uploaded batches
- Receipt-image processing

---

8. Shared Human-in-the-Loop Requirement

Every challenge must include a review screen.

Before the final result is confirmed, the user must be able to:

- View the extracted data
- View warnings and conflicts
- View the AI's recommendation
- View the supporting explanation
- Edit at least one extracted value
- Approve or reject the final output

The final output should indicate whether it was approved.

Example:

```
{
  "review_status": "approved",
  "reviewed_by": "user",
  "changes_made": true
}
```

Teams do not need to build authentication or store a real reviewer identity.

---

9. Optional Context Memory

Context memory is optional and may contribute to the Innovation score.

A solution with context memory may retain relevant information across interactions.

Examples include:

- A dispatcher sends a second update about the same trip.
- A citizen provides a landmark after being asked for clarification.
- A student changes available study hours.

- A business owner corrects a merchant category.

The user must be able to update or override stored context. The application should not continue using information after the user has corrected or removed it.

---

10. Synthetic Data Pack

The organisers will provide synthetic data for all four challenges.

The data pack contains:

- Fleet manifests, dispatcher messages, driver updates and delivery notes
- Utility reports in multiple South African languages
- Academic briefs and assessment instructions
- Expense, receipt and VAT records
- Incomplete and ambiguous cases
- Duplicate and conflicting cases
- Organiser-only answer keys

Teams are not required to process every supplied case.

Each team must demonstrate at least:

- One normal case
- One incomplete or ambiguous case
- One risk, duplicate, conflict, or anomaly case

Judges may test the application using one additional supplied case that the team has not demonstrated.

---

11. Evaluation Rubric

All four challenges will be evaluated using the same 100-mark rubric.

Category	Marks	Excellent solution

AI Reasoning and Intelligence	30	Goes beyond extraction by identifying ambiguity, asking relevant clarification questions, making justified recommendations and explaining important decisions.
Accuracy and Reliability	20	Produces valid structured output, validates inputs, handles missing or conflicting information, avoids invented values and communicates uncertainty responsibly.
Workflow and Problem Solving	20	Delivers a complete workflow covering input, analysis, validation, recommendation, human review and approved final output.
User Experience	15	Provides a clean, responsive and intuitive interface with editable AI output, visible warnings and understandable results.
Technical Quality	10	Demonstrates secure model integration, structured architecture, schema validation, deterministic business rules, error handling and maintainable code.
Innovation	5	Adds a useful enhancement such as context memory, multilingual support, visualisation, automation or another challenge-relevant capability.
Total	**100**	

11.1 Judging guidance

Judges should assess meaning rather than exact wording.

Teams should not be penalised when:

- A missing field is correctly returned as `null`.
- A summary uses different but accurate wording.

- A category name differs but has the same meaning.
- A clarification question is phrased differently.
- An estimated plan differs while remaining reasonable and properly labelled.

Teams should lose accuracy or reliability marks when they:

- Invent important facts
- Hide uncertainty
- Produce invalid calculations
- Recommend unsupported actions
- Fail to identify obvious conflicts
- Automatically approve high-risk outputs
- Expose API credentials
- Fail when given an unexpected input

11.2 Innovation features

Optional features are assessed within the five Innovation marks. They do not increase the total above 100.

A team can receive a strong overall score without implementing an optional feature if its core workflow is accurate, reliable and well executed.

---

12. Minimum Submission Requirements

Each team must submit:

- A functional MVP
- A link to its source-code repository
- A deployed application or locally runnable version
- A README containing setup instructions
- The name of the open-weight model used
- A list of external libraries or services used
- At least three sample inputs and outputs
- A basic architecture diagram
- A short limitations section
- A short live demonstration or presentation

The README should explain:

- How to run the application

- How to configure the AI provider
- How structured output is validated
- Which calculations or rules are handled in code
- What happens when information is missing
- What happens when the AI service fails
- Which optional innovation features were implemented

---

13. Demonstration Requirements

The live demonstration should show:

1. The selected challenge and target user.
2. One normal input.
3. One incomplete, conflicting, duplicate, or anomalous input.
4. The structured AI output.
5. The AI's recommendation and reasoning.
6. A user editing or correcting the output.
7. The user approving or rejecting the result.
8. The final structured record.
9. One known limitation or failure case.

The organisers may provide one additional test case during judging.

---

14. Recommended 10-Hour Schedule

Time	Recommended activity
Hour 1	Select a challenge, understand the supplied data and define one core workflow.
Hours 2–3	Design the interface, structured-output schema, prompt and validation rules.

Hours 4–5	Implement the AI integration and extraction workflow.
Hour 6	Add ambiguity detection, recommendations and explanations.
Hour 7	Add the human review and approval step.
Hour 8	Add error handling and one optional enhancement if time permits.
Hour 9	Test normal, incomplete and unexpected inputs.
Hour 10	Deploy, complete the README and rehearse the demonstration.

---

15. Scope Management

This is a 10-hour Buildathon.

Teams should prioritise one complete and reliable workflow over a large collection of unfinished features.

A suitable submission may consist of:

One focused user interface

↓

One open-weight AI integration

↓

One structured-output schema

↓

One reasoning and recommendation step

↓

One human review screen

↓

One approved final result

Teams do not need:

- Production authentication
- Multiple user roles
- Production databases
- Real external-system integrations
- Mobile and web applications
- Advanced cloud infrastructure
- Large-scale machine-learning training
- A fully autonomous agent
- Every optional feature

---

16. Final Principle

The strongest solutions will not be those that generate the most text or use the largest number of technologies.

The strongest solutions will:

- Understand the problem
- Produce accurate structured information
- Detect uncertainty
- Reason about the available evidence
- Recommend a useful next action
- Explain that recommendation
- Keep a human in control
- Deliver a complete and usable workflow within 10 hours